

Code Review Report

Language: python | Review mode: hybrid | Model: llama-3.3-70b-versatile | Generated: 2026-08-13 17:28 UTC

Overall Assessment

The code review has identified 12 findings with a risk score of 45, indicating a high level of risk. The majority of the findings are critical, highlighting the need for immediate attention to prevent potential security breaches.

Merge readiness: Not Ready

Code health: 0/100

Findings Summary

Total findings: 12

Risk score: 45

Severity breakdown: Low 0, Medium 0, High 3, Critical 9

Type	Severity	Line	Message	Recommendation
Hardcoded Password	High	1	password = "admin123"	Store passwords in environment variables.
Hardcoded API Key	High	3	api_key = "123456789"	Store secrets securely.
SQL Injection	Critical	7	query = "SELECT * FROM users WHERE name='" + name + "'"	Use parameterized queries.
Dangerous Dynamic Execution	Critical	9	eval(input())	Avoid dynamic code execution.
Dangerous Dynamic Execution	Critical	11	exec("print('Hello')")	Avoid dynamic code execution.
Command Injection	Critical	15	subprocess.run("dir", shell=True)	Do not use shell=True with user input.
Security	Critical	1	Hardcoded sensitive information (password)	Store sensitive information securely using environment variables or a secrets manager.
Security	Critical	2	Hardcoded sensitive information (api_key)	Store sensitive information securely using environment variables or a secrets manager.
Security	Critical	5	SQL injection vulnerability due to string concatenation	Use parameterized queries or prepared statements to prevent SQL injection.
Security	Critical	6	Use of eval() function with user input	Avoid using eval() with user input; instead, use a safer alternative, such as ast.literal_eval() or a parsing library.

Type	Severity	Line	Message	Recommendation
Security	Critical	7	Use of exec() function	Avoid using exec(); instead, use a safer alternative, such as the subprocess module with a whitelist of allowed commands.
Security	High	9	Use of subprocess with shell=True	Avoid using shell=True; instead, pass the command and arguments as a list to prevent shell injection.

Remediation Roadmap

Finding	Severity	Impact	Suggested Action
SQL Injection	Critical	High risk of data breach	Replace string concatenation with parameterized queries
Dangerous Dynamic Execution	Critical	High risk of code injection attacks	Replace dynamic execution with static, parameterized code
Command Injection	Critical	High risk of executing malicious commands	Replace shell=True with shell=False and pass the command as a list
Security	Critical	High risk of security breaches	Store sensitive information securely using environment variables or a secrets manager
Hardcoded Password	High	High risk of password exposure	Store passwords in environment variables
Hardcoded API Key	High	High risk of API key exposure	Store API keys securely using environment variables or a secrets manager
Security	High	High risk of security breaches	Avoid using eval() with user input and replace with safer alternatives

Source Code Snapshot

```
password = "admin123"

api_key = "123456789"

name = input()

query = "SELECT * FROM users WHERE name='" + name + "'"

eval(input())

exec("print('Hello')")

import subprocess

subprocess.run("dir", shell=True)
```